Algorytmy macierzowe, ćwiczenie 1

Ivan Lazorenko, Bartłomiej Wiśniewski

14 października 2026

1 Dane techniczne

Obliczenia wykonano w następującym środowisku:

1. procesor AMD Ryzen 5 5500, 16 GB pamięci RAM, karta graficzna AMD Radeon RX 6700 XT (12 GB VRAM),
2. system operacyjny Windows 11 Home 64-bit,
3. `Python 3.12.6` z kompilatorem `Cython` oraz `GNU Octave`.

Wykorzystano kompilator `Cython`, który tłumaczy kod Pythona do języka C i kompiluje go do natywnej biblioteki kodu maszynowego (`.pyd`). Dzięki temu kod nie jest interpretowany linia po linii w czasie działania, tylko wykonywany bezpośrednio przez procesor.

2 Metoda Bineta

Algorytm Bineta dzieli macierze wejściowe A i B na cztery mniejsze podmacierze i wykonuje rekurencyjnie 8 mnożeń oraz 4 dodawania bloków:

$$C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{pmatrix}$$

```python
def binet(A: np.ndarray, B: np.ndarray) -> tuple[np.ndarray, int, int]:
    rows, common = A.shape
    cols = B.shape[1]

    if rows == 1 or common == 1 or cols == 1:
        C = np.zeros((rows, cols))
        multiplications = 0
        additions = 0

        for i in range(rows):
            for j in range(cols):
                C[i, j] = A[i, 0] * B[0, j]
                multiplications += 1
                for k in range(1, common):
                    C[i, j] += A[i, k] * B[k, j]
                    multiplications += 1
                    additions += 1

        return C, multiplications, additions

    mid_rows = rows // 2
    mid_common = common // 2
    mid_cols = cols // 2

    A11, A12 = A[:mid_rows, :mid_common], A[:mid_rows, mid_common:]
    A21, A22 = A[mid_rows:, :mid_common], A[mid_rows:, mid_common:]
    B11, B12 = B[:mid_common, :mid_cols], B[:mid_common, mid_cols:]
    B21, B22 = B[mid_common:, :mid_cols], B[mid_common:, mid_cols:]

    P1, mul1, add1 = binet(A11, B11)
    P2, mul2, add2 = binet(A12, B21)
    C11 = P1 + P2

    P3, mul3, add3 = binet(A11, B12)
```

$[C, \text{mul}, \text{add}] = \text{Binet}(A, B)$

if $\text{rows}(A) = 1$ **or** $\text{cols}(A) = 1$ **or** $\text{cols}(B) = 1$

$$C_{i,j} := \sum_{k=1}^{\text{cols}(A)} A_{i,k} \cdot B_{k,j}$$
$$\text{mul} := \text{rows}(A) \cdot \text{cols}(A) \cdot \text{cols}(B)$$
$$\text{add} := \text{rows}(A) \cdot (\text{cols}(A) - 1) \cdot \text{cols}(B)$$

else

$$A \to \left(\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array}\right), \quad B \to \left(\begin{array}{c|c} B_{11} & B_{12} \\ \hline B_{21} & B_{22} \end{array}\right)$$

$$[P_1, m_1, a_1] := \text{Binet}(A_{11}, B_{11}), \quad [P_2, m_2, a_2] := \text{Binet}(A_{12}, B_{21}), \quad C_{11} := P_1 + P_2$$
$$[P_3, m_3, a_3] := \text{Binet}(A_{11}, B_{12}), \quad [P_4, m_4, a_4] := \text{Binet}(A_{12}, B_{22}), \quad C_{12} := P_3 + P_4$$
$$[P_5, m_5, a_5] := \text{Binet}(A_{21}, B_{11}), \quad [P_6, m_6, a_6] := \text{Binet}(A_{22}, B_{21}), \quad C_{21} := P_5 + P_6$$
$$[P_7, m_7, a_7] := \text{Binet}(A_{21}, B_{12}), \quad [P_8, m_8, a_8] := \text{Binet}(A_{22}, B_{22}), \quad C_{22} := P_7 + P_8$$
$$\text{mul} := \sum_{i=1}^{8} m_i, \qquad \text{add} := \sum_{i=1}^{8} a_i + \text{size}(C)$$

$$C \leftarrow \left(\begin{array}{c|c} C_{11} & C_{12} \\ \hline C_{21} & C_{22} \end{array}\right)$$

endif

Rysunek 1: Pseudokod algorytmu Bineta.

```
    P4, mul4, add4 = binet(A12, B22)
    C12 = P3 + P4

    P5, mul5, add5 = binet(A21, B11)
    P6, mul6, add6 = binet(A22, B21)
    C21 = P5 + P6

    P7, mul7, add7 = binet(A21, B12)
    P8, mul8, add8 = binet(A22, B22)
    C22 = P7 + P8

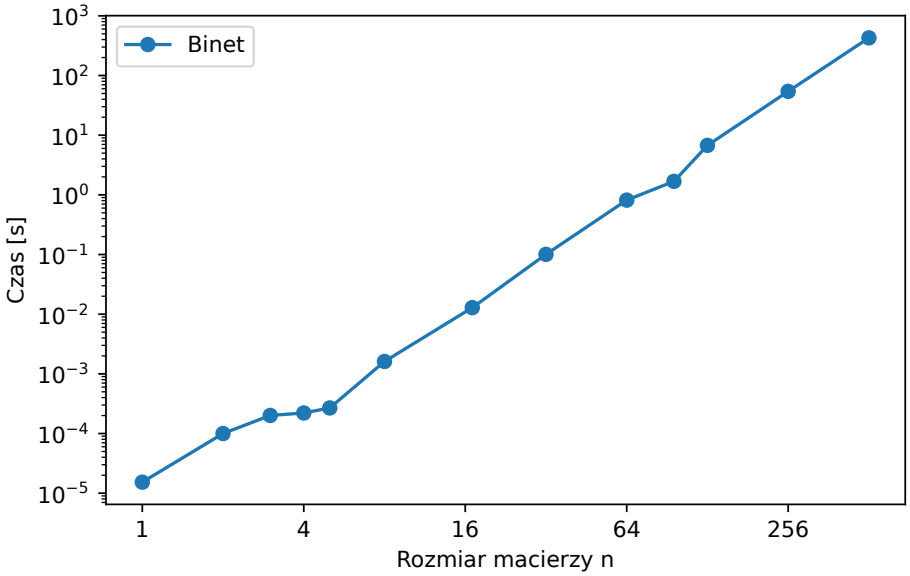
    C = np.block([[C11, C12], [C21, C22]])

    multiplications = mul1 + mul2 + mul3 + mul4 + mul5 + mul6 + mul7 + mul8
    additions = (
        add1 + add2 + add3 + add4 + add5 + add6 + add7 + add8
        + C11.size + C12.size + C21.size + C22.size
    )
    return C, multiplications, additions
```

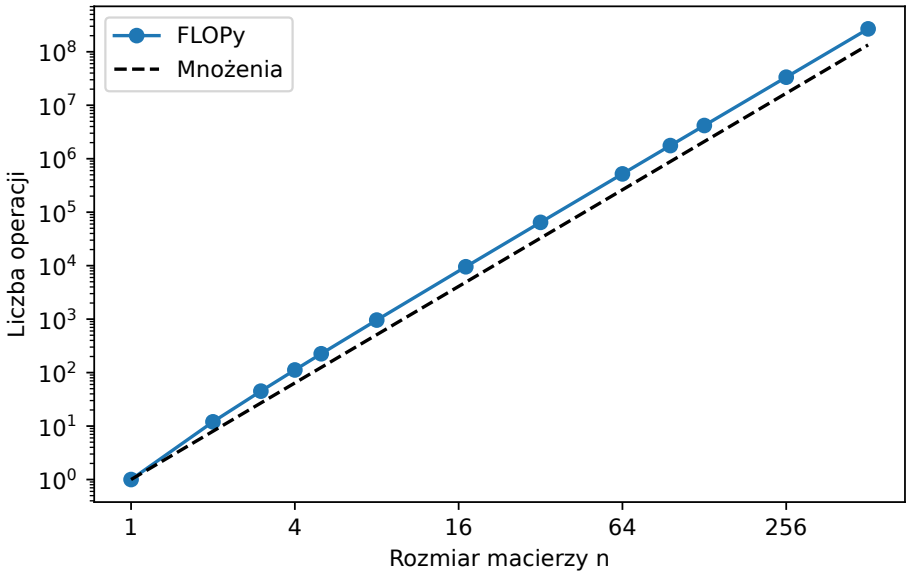
Wykaz 1: Implementacja metody Bineta.

Tabela 1: Wyniki czasowe i operacyjne (metoda Bineta)

Rozmiar n	Czas [s]	Mnożenia	Dodawania	FLOPy
1	0.000015	1	0	1
2	0.000100	8	4	12
3	0.000201	27	18	45
4	0.000221	64	48	112
5	0.000268	125	100	225
8	0.001611	512	448	960
17	0.012865	4913	4624	9537
32	0.100570	32768	31744	64512
64	0.815996	262144	258048	520192
96	1.682287	884736	875520	1760256
128	6.744274	2097152	2080768	4177920
256	54.217998	16777216	16711680	33488896
512	427.090985	134217728	133955584	268173312



Rysunek 2: Czas działania metody Bineta.



Rysunek 3: Liczba operacji zmiennoprzecinkowych metody Bineta.

3 Algorytm Strassena

Algorytm Strassena redukuje liczbę rekurencyjnych mnożeń z 8 do 7 kosztem 18 dodawań i odejmowań podmacierzy. Dla nieparzystych rozmiarów n funkcja `strassen_odd` odcina ostatni wiersz i kolumnę, które są mnożone bezpośrednio przez `multiply_direct`.

$$C = \begin{pmatrix} P_1 + P_4 - P_5 + P_7 & P_3 + P_5 \\ P_2 + P_4 & P_1 - P_2 + P_3 + P_6 \end{pmatrix}$$

```python
def strassen(A: np.ndarray, B: np.ndarray) -> tuple[np.ndarray, int, int]:
    n = A.shape[0]

    if n == 1:
        return np.array([[A[0, 0] * B[0, 0]]]), 1, 0

    if n % 2 == 1:
        return strassen_odd(A, B)

    mid = n // 2
    A11, A12 = A[:mid, :mid], A[:mid, mid:]
    A21, A22 = A[mid:, :mid], A[mid:, mid:]
    B11, B12 = B[:mid, :mid], B[:mid, mid:]
    B21, B22 = B[mid:, :mid], B[mid:, mid:]

    P1, mul1, add1 = strassen(A11 + A22, B11 + B22)
    P2, mul2, add2 = strassen(A21 + A22, B11)
    P3, mul3, add3 = strassen(A11, B12 - B22)
    P4, mul4, add4 = strassen(A22, B21 - B11)
    P5, mul5, add5 = strassen(A11 + A12, B22)
    P6, mul6, add6 = strassen(A21 - A11, B11 + B12)
    P7, mul7, add7 = strassen(A12 - A22, B21 + B22)

    C11 = P1 + P4 - P5 + P7
    C12 = P3 + P5
    C21 = P2 + P4
    C22 = P1 - P2 + P3 + P6
    C = np.block([[C11, C12], [C21, C22]])

    multiplications = mul1 + mul2 + mul3 + mul4 + mul5 + mul6 + mul7
    additions = add1 + add2 + add3 + add4 + add5 + add6 + add7 + 18 * mid * mid
    return C, multiplications, additions


def strassen_odd(A: np.ndarray, B: np.ndarray) -> tuple[np.ndarray, int, int]:
    A0, a = A[:-1, :-1], A[:-1, -1:]
    r, alpha = A[-1:, :-1], A[-1:, -1:]
```

$[C, \text{mul}, \text{add}] = \text{Strassen}(A, B)$

if $n = 1$

$$C := \left(A_{0,0} \cdot B_{0,0}\right), \quad \text{mul} := 1, \quad \text{add} := 0$$

else if $n \pmod 2 = 1$

$$[C, \text{mul}, \text{add}] := \text{StrassenOdd}(A, B)$$

else

$$A \to \left(\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array}\right), \quad B \to \left(\begin{array}{c|c} B_{11} & B_{12} \\ \hline B_{21} & B_{22} \end{array}\right)$$

$$\begin{aligned}
&[P_1, m_1, a_1] := \text{Strassen}(A_{11} + A_{22}, B_{11} + B_{22}) \\
&[P_2, m_2, a_2] := \text{Strassen}(A_{21} + A_{22}, B_{11}) \\
&[P_3, m_3, a_3] := \text{Strassen}(A_{11}, B_{12} - B_{22}) \\
&[P_4, m_4, a_4] := \text{Strassen}(A_{22}, B_{21} - B_{11}) \\
&[P_5, m_5, a_5] := \text{Strassen}(A_{11} + A_{12}, B_{22}) \\
&[P_6, m_6, a_6] := \text{Strassen}(A_{21} - A_{11}, B_{11} + B_{12}) \\
&[P_7, m_7, a_7] := \text{Strassen}(A_{12} - A_{22}, B_{21} + B_{22})
\end{aligned}$$

$$\begin{aligned}
C_{11} &:= P_1 + P_4 - P_5 + P_7, & C_{12} &:= P_3 + P_5 \\
C_{21} &:= P_2 + P_4, & C_{22} &:= P_1 - P_2 + P_3 + P_6 \\
\text{mul} &:= \sum_{i=1}^{7} m_i, & \text{add} &:= \sum_{i=1}^{7} a_i + 18 \cdot \left(\frac{n}{2}\right)^2
\end{aligned}$$

$$C \leftarrow \left(\begin{array}{c|c} C_{11} & C_{12} \\ \hline C_{21} & C_{22} \end{array}\right)$$

endif

Rysunek 4: Pseudokod algorytmu Strassena.

$[C, \text{mul}, \text{add}] = \text{StrassenOdd}(A, B)$

$$A \to \left(\begin{array}{c|c} A_0 & a \\ \hline r & \alpha \end{array}\right), \quad B \to \left(\begin{array}{c|c} B_0 & b \\ \hline s & \beta \end{array}\right)$$

$$\begin{aligned}
[P_1, m_1, a_1] &:= \text{Strassen}(A_0, B_0), & [P_2, m_2, a_2] &:= \text{MultiplyDirect}(a, s) \\
[P_3, m_3, a_3] &:= \text{MultiplyDirect}(A_0, b), & [P_4, m_4, a_4] &:= \text{MultiplyDirect}(a, \beta) \\
[P_5, m_5, a_5] &:= \text{MultiplyDirect}(r, B_0), & [P_6, m_6, a_6] &:= \text{MultiplyDirect}(\alpha, s) \\
[P_7, m_7, a_7] &:= \text{MultiplyDirect}(r, b), & [P_8, m_8, a_8] &:= \text{MultiplyDirect}(\alpha, \beta) \\
C_{11} &:= P_1 + P_2, \quad C_{12} := P_3 + P_4, & C_{21} &:= P_5 + P_6, \quad C_{22} := P_7 + P_8 \\
\text{mul} &:= \sum_{i=1}^{8} m_i, & \text{add} &:= \sum_{i=1}^{8} a_i + \text{size}(C)
\end{aligned}$$

$$C \leftarrow \left(\begin{array}{c|c} C_{11} & C_{12} \\ \hline C_{21} & C_{22} \end{array}\right)$$

Rysunek 5: Pseudokod procedury `StrassenOdd`.

```
    B0, b = B[:-1, :-1], B[:-1, -1:]
    s, beta = B[-1:, :-1], B[-1:, -1:]

    P1, mul1, add1 = strassen(A0, B0)
    P2, mul2, add2 = multiply_direct(a, s)
    P3, mul3, add3 = multiply_direct(A0, b)
    P4, mul4, add4 = multiply_direct(a, beta)
    P5, mul5, add5 = multiply_direct(r, B0)
    P6, mul6, add6 = multiply_direct(alpha, s)
    P7, mul7, add7 = multiply_direct(r, b)
    P8, mul8, add8 = multiply_direct(alpha, beta)

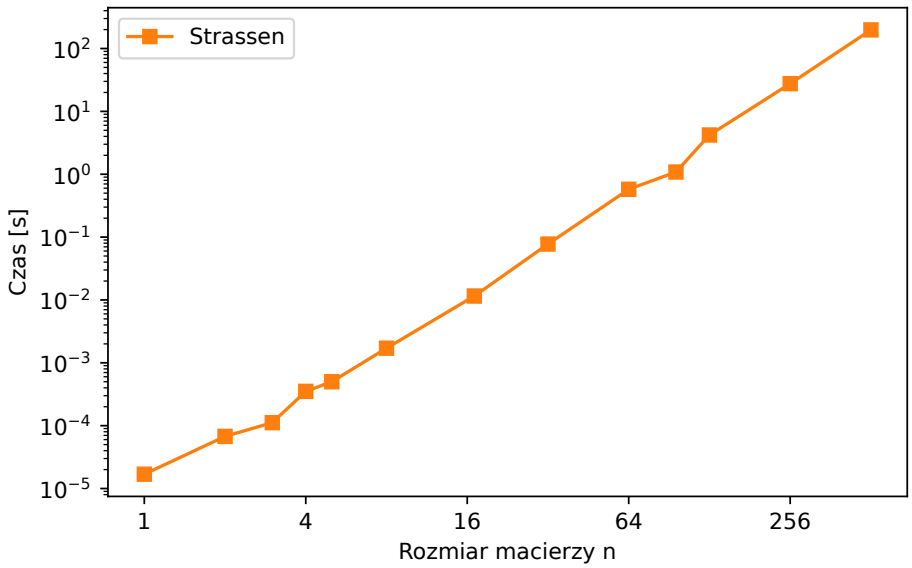
    C11, C12 = P1 + P2, P3 + P4
    C21, C22 = P5 + P6, P7 + P8
    C = np.block([[C11, C12], [C21, C22]])

    multiplications = mul1 + mul2 + mul3 + mul4 + mul5 + mul6 + mul7 + mul8
    additions = (
        add1 + add2 + add3 + add4 + add5 + add6 + add7 + add8
        + C11.size + C12.size + C21.size + C22.size
    )
    return C, multiplications, additions
```

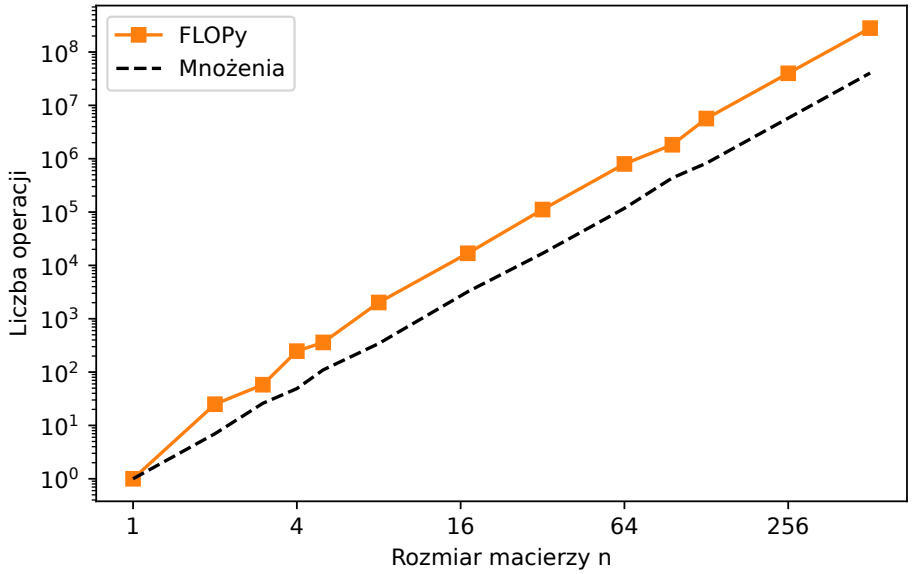
Wykaz 2: Implementacja metody Strassena oraz procedury dla wymiarów nieparzystych.

Tabela 2: Wyniki czasowe i operacyjne (metoda Strassena)

Rozmiar n	Czas [s]	Mnożenia	Dodawania	FLOPy
1	0.000002	1	0	1
2	0.000034	7	18	25
3	0.000061	26	32	58
4	0.000226	49	198	247
5	0.000271	110	250	360
8	0.001624	343	1674	2017
17	0.012249	3218	13654	16872
32	0.083366	16807	94698	111505
64	0.584557	117649	681318	798967
96	1.044563	436982	1390106	1827088
128	4.023335	823543	4842954	5666497
256	29.379938	5764801	34195590	39960391
512	210.828233	40353607	240548778	280902385



Rysunek 6: Czas działania metody Strassena.



Rysunek 7: Liczba operacji zmiennoprzecinkowych metody Strassena.

4 Algorytm AlphaTensor

Algorytm AlphaTensor dzieli macierze po dopełnieniu zerami na bloki 4×5 oraz 5×5 i wykonuje 76 mnożeń rekurencyjnych zamiast 100. Mniejsze podmacierze są mnożone bezpośrednio przez `multiply_direct`.

$[C, \text{mul}, \text{add}] = \text{AlphaTensor}(A, B)$

if $\text{rows}(A) < 4$ **or** $\text{cols}(A) < 5$ **or** $\text{cols}(B) < 5$

$[C, \text{mul}, \text{add}] := \text{MultiplyDirect}(A, B)$

---

else

$\tilde{A} := \text{Pad}(A, 4, 5), \quad \tilde{B} := \text{Pad}(B, 5, 5)$

$\tilde{A} \to (A_{i,j})_{\substack{0 \le i < 4 \\ 0 \le j < 5}}, \quad \tilde{B} \to (B_{j,k})_{\substack{0 \le j < 5 \\ 0 \le k < 5}}$

---

for $r = 0$ **to** 75 **do**

$[L_r, \text{op}_{L,r}] := \text{Combine}(\{A_{i,j}\}, U_r), \qquad [R_r, \text{op}_{R,r}] := \text{Combine}(\{B_{j,k}\}, V_r)$

$[P_r, m_r, a_r] := \text{AlphaTensor}(L_r, R_r)$

endfor

for $idx = 0$ **to** 19 **do**

$i := \lfloor idx/5 \rfloor, \quad k := idx \pmod 5, \quad [C_{i,k}, \text{op}_{C,idx}] := \text{Combine}(\{P_r\}, W_{idx})$

endfor

$\text{mul} := \sum_{r=0}^{75} m_r, \quad \text{add} := \sum_{r=0}^{75} \left(a_r + \text{op}_{L,r} + \text{op}_{R,r}\right) + \sum_{idx=0}^{19} \text{op}_{C,idx}$

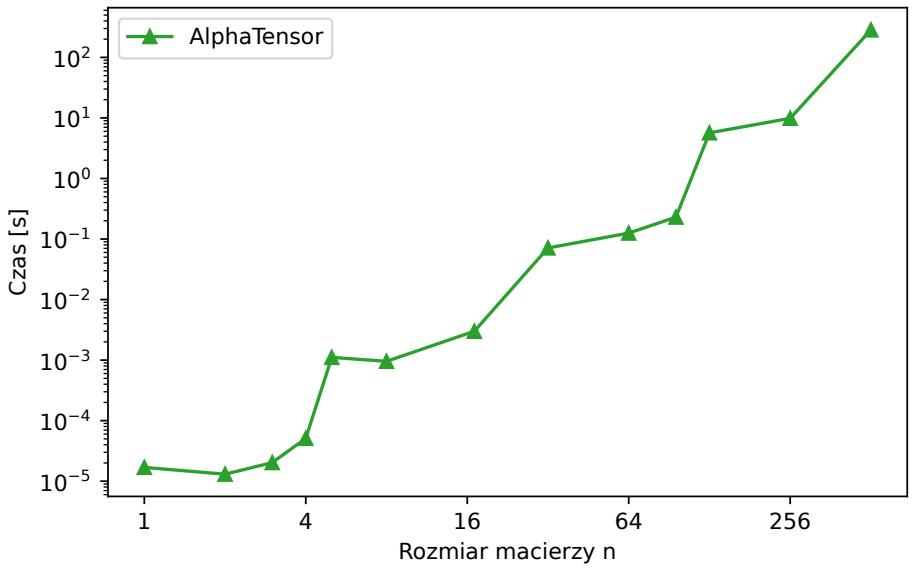
$\tilde{C} \leftarrow (C_{i,k})_{\substack{0 \le i < 4 \\ 0 \le k < 5}}, \quad C \leftarrow \tilde{C}[0 : \text{rows}(A), 0 : \text{cols}(B)]$

endif

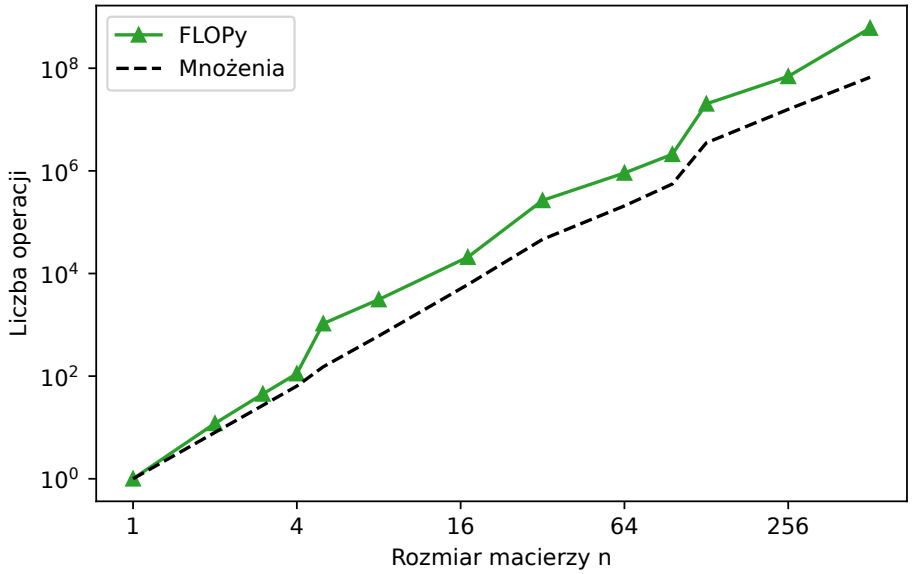
Rysunek 8: Pseudokod algorytmu AlphaTensor.

Tabela 3: Wyniki czasowe i operacyjne (metoda AlphaTensor)

Rozmiar n	Czas [s]	Mnożenia	Dodawania	FLOPy
1	0.000017	1	0	1
2	0.000013	8	4	12
3	0.000020	27	18	45
4	0.000051	64	48	112
5	0.001110	152	904	1056
8	0.000954	608	2500	3108
17	0.002994	6080	14764	20844
32	0.071194	46208	219386	265594
64	0.125217	207936	701706	909642
96	0.229820	554496	1547296	2101792
128	5.669803	3511808	16692328	20204136
256	9.849167	15803136	53464600	69267736
512	284.307233	66724352	538238234	604962586



Rysunek 9: Czas działania metody AlphaTensor.

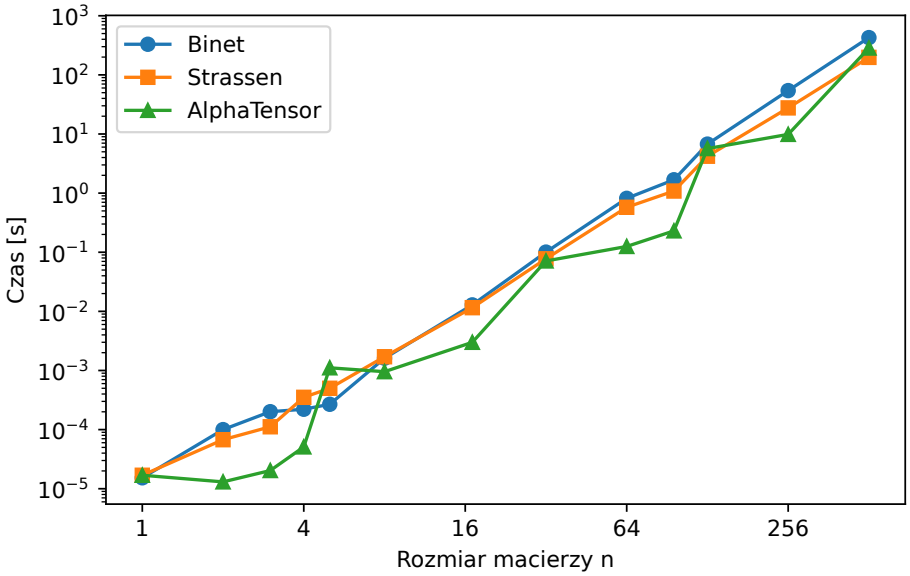


Rysunek 10: Liczba operacji zmiennoprzecinkowych metody AlphaTensor.

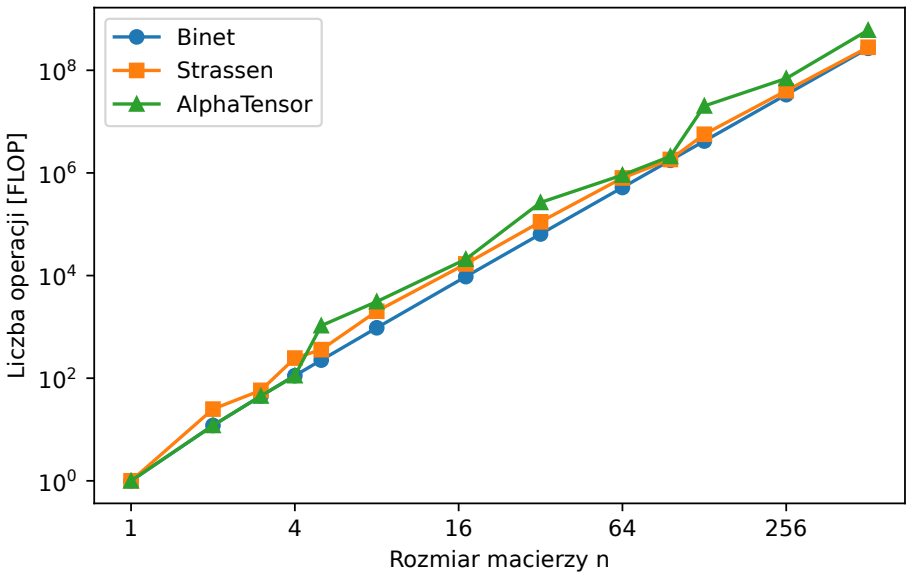
5 Oszacowanie złożoności i porównanie wyników

Złożoność obliczeniowa badanych algorytmów wynosi:

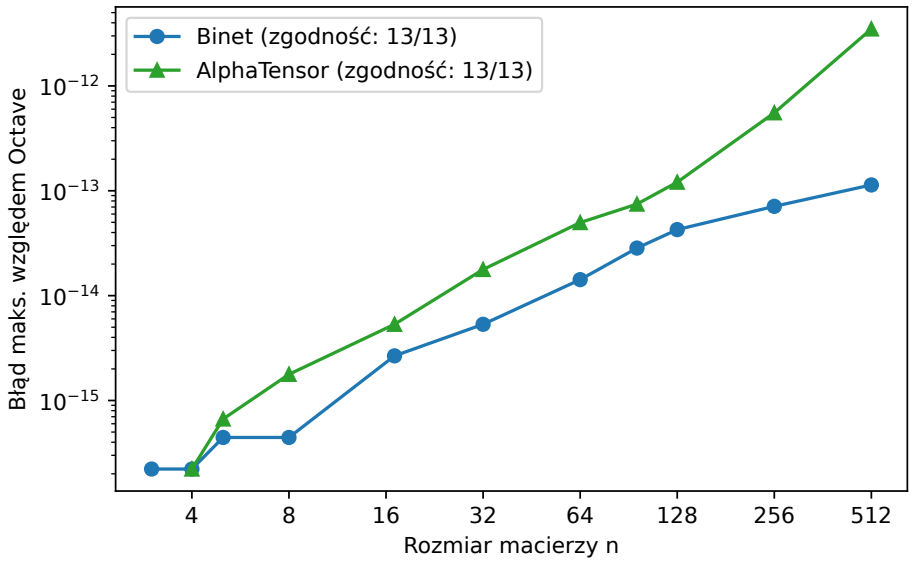
- **Binet:** 8 wywołań rekurencyjnych dla podmacierzy rozmiaru $n/2$ daje złożoność $\mathcal{O}(n^{\log_2 8}) = \mathcal{O}(n^3)$.
- **Strassen:** 7 wywołań rekurencyjnych dla podmacierzy rozmiaru $n/2$ daje złożoność $\mathcal{O}(n^{\log_2 7}) \approx \mathcal{O}(n^{2.81})$.
- **AlphaTensor:** 76 wywołań rekurencyjnych zamiast 100 przy podziale blokowym $4 \times 5 \times 5$ daje złożoność około $\mathcal{O}(n^{2.81})$ (w praktyce dla potęg dwójki wykładnik czasu i operacji jest zbliżony do $n^{2.9}$ przez dopełnianie zerami do wielokrotności 5).



Rysunek 11: Porównanie czasu działania badanych algorytmów.



Rysunek 12: Porównanie liczby operacji zmiennoprzecinkowych.



Rysunek 13: Maksymalny błąd bezwzględny względem GNU Octave (zgodność wyników dla wszystkich badanych rozmiarów).